

# Chapter 4 Summary

## Summary in Arabic

### الفصل 4: نمذجة العلاقة الموسعة بين الكيانات (Enhanced Entity-Relationship Modeling - EER)

#### مقدمة

نموذج EER هو امتداد لنموذج ER (Entity-Relationship) الذي تم تناوله في الفصل السابق. يُعرف بـ النموذج الموسع أو المُحسّن، ويوفر مفاهيم إضافية تسمح بتمثيل البيانات بشكل أكثر دقة وتعقيداً، خصوصاً في النظم الكبيرة أو المعقدة.

#### ✓ ما الذي يضيفه نموذج EER إلى نموذج ER؟

- التخصص (Specialization) والتعميم (Generalization)
- الطبقات الفرعية (Subclasses) والطبقات العليا (Superclasses)
- الوراثة (Inheritance)
- القيود (Constraints) على التخصص والتعميم
- الأصناف (Categories) أو أنواع الاتحاد (Union Types)

### 1. الطبقات الفرعية والطبقات العليا (Subclasses and Superclasses)

#### المفهوم

- من الممكن أن تكون كيانات من نوع معين (مثلاً: EMPLOYEE) تندرج تحت مجموعات فرعية ذات خصائص إضافية.
- هذه المجموعات الفرعية تُسمى الطبقات الفرعية (Subclasses).
- بينما يُسمى النوع الأصلي الطبقة العليا (Superclass).

#### مثال توضيحي

- الكيان: EMPLOYEE (موظف)
- طبقات فرعية منه:
- SECRETARY (سكرتير)

- **ENGINEER** (مهندس)
- **TECHNICIAN** (فني)
- **MANAGER** (مدير)
- **SALARIED\_EMPLOYEE** (موظف راتبه شهري)
- **HOURLY\_EMPLOYEE** (موظف يُدفع له بالساعة)

🔦 لاحظ أن كل واحد من هؤلاء هو نوع من أنواع **EMPLOYEE**.

## علاقة IS-A (هي-من-نوع)

- تُعرف العلاقة بين الطبقة العليا والفرعية بعلاقة **IS-A**:
- **SECRETARY IS-A EMPLOYEE**
- **ENGINEER IS-A EMPLOYEE**
- هذا يعني أن الكيان في الطبقة الفرعية هو نفس الكيان في الطبقة العليا، لكنه يؤدي دورًا أو وظيفة محددة.

### ملاحظات مهمة

- لا يمكن لكيان أن يكون في الطبقة الفرعية فقط دون أن يكون في الطبقة العليا.
- يمكن لكيان أن ينتمي إلى أكثر من طبقة فرعية في نفس الوقت.
- مثال: موظف قد يكون مديرًا ومهندسًا وموظفًا راتبياً في آن واحد.
- ليس من الضروري أن يكون كل كيان في الطبقة العليا عضواً في إحدى الطبقات الفرعية.

## 2. التخصص (Specialization)

### التعريف

عملية تحديد مجموعات فرعية من كيان في الطبقة العليا بناءً على خصائص أو تمييز معين.

### أمثلة

1. تخصص **EMPLOYEE** إلى:
  - بناءً على نوع الوظيفة {SECRETARY, ENGINEER, TECHNICIAN}.
2. تخصص آخر لنفس **EMPLOYEE** إلى:
  - بناءً على طريقة الدفع {SALARIED\_EMPLOYEE, HOURLY\_EMPLOYEE}.

✅ يمكن وجود أكثر من تخصص لنفس الطبقة العليا.

## السمات والعلاقات الخاصة (Specific Attributes & Relationships)

- يمكن للطبقة الفرعية أن تمتلك:
- سمات خاصة بها (لا تُورث من الأعلى).
- مثال: السكرتير لديه سمة سرعة الكتابة (TypingSpeed).
- علاقات خاصة بها.

- مثال: HOURLY\_EMPLOYEE قد يكون له علاقة BELONGS\_TO مع جدول الدوام.

### 3. التعميم (Generalization)

#### التعريف

هي العملية العكسية للتخصص:

- نبدأ بعدة كيانات (مثل: CAR، TRUCK) ونجمعها في كيان أعلى مستوى (مثل: VEHICLE).
- أي: تجميع الكيانات المشتركة في خصائص إلى كيان عام.

#### مثال

- **VEHICLE** يتم تعميمهما إلى **CAR** و **TRUCK**.
- إذا: VEHICLE هو الطبقة العليا، و CAR و TRUCK هما الطبقتان الفرعيتان.

💡 يمكن النظر إلى الأمر من زاويتين:

- من الأعلى إلى الأسفل: تخصص (Specialization)
- من الأسفل إلى الأعلى: تعميم (Generalization)

#### ملاحظة مهمة

- في الممارسة العملية، يتم استخدام كلا العمليتين معًا.
- لا يوجد فرق جوهري في التمثيل البياني، لذلك لا يُشترط رسم سهم يشير إلى التخصص أو التعميم.

### 4. الوراثة (Inheritance)

#### المفهوم

- أي كيان في الطبقة الفرعية يرث:
- جميع السمات (Attributes) من الطبقة العليا.
- جميع العلاقات (Relationships) من الطبقة العليا.

#### مثال

- كيان **SECRETARY** يرث من **EMPLOYEE**:
- السمات: Name، SSN، Address، ...
- العلاقات: WORKS\_FOR، MANAGES، ...

✅ هذا يعني أن كل سكرتير لديه اسم ورقم تعريف، إلخ.

## 5. أنواع التخصص (Types of Specialization)

| النوع                                 | الوصف                                                                          | مثال                                                     |
|---------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------|
| مُعرّف بالشرط (Predicate-defined)     | يتم تحديد العضوية في الطبقة الفرعية بناءً على شرط (مثلاً: قيمة سمة).           | ENGINEER إذا كانت JobType = "Engineer"                   |
| مُعرّف بالسمة (Attribute-defined)     | الشرط مبني على سمة معينة في الطبقة العليا. تُكتب السمة عند خط الربط.           | JobType {SECRETARY, ENGINEER, TECHNICIAN} تُستخدم لتحديد |
| مُعرّف من قبل المستخدم (User-defined) | لا يوجد شرط تلقائي، بل يُضاف الكيان يدوياً إلى الطبقة الفرعية بواسطة المستخدم. | مثلاً: إضافة موظف إلى فئة "موظف متميز" حسب تقييم المدير  |

## 6. القيود على التخصص والتعميم (Constraints)

### أ. قيد التباعد (Disjointness Constraint)

- هل يمكن للكيان أن يكون في أكثر من طبقة فرعية؟
- ✓ منفصل (Disjoint - d): لا يمكن أن يكون في أكثر من طبقة فرعية.
- مثال: الموظف لا يمكن أن يكون سكرتيراً وفنياً في نفس الوقت.
- ✗ متداخل (Overlapping - o): يمكن أن يكون في أكثر من طبقة فرعية.
- مثال: موظف يمكن أن يكون مديراً ومهندساً معاً.

### ب. قيد الاكتمال (Completeness Constraint)

- هل كل كيان في الطبقة العليا يجب أن يكون في إحدى الطبقات الفرعية؟
- ✓ كلي (Total - خط مزدوج): كل كيان في الطبقة العليا يجب أن ينتمي إلى إحدى الطبقات الفرعية.
- مثال: كل EMPLOYEE يجب أن يكون إما HOURLY أو SALARIED.
- ✗ جزئي (Partial - خط واحد): يُسمح للكيان ألا ينتمي إلى أي طبقة فرعية.
- مثال: بعض الموظفين قد لا يكونون في أي تخصص (مثل: موظف جديد لم يُحدد نوعه بعد).

### ✓ الأنواع الأربعة للقيود:

| التباعد | الاكتمال | الوصف                                                                                  |
|---------|----------|----------------------------------------------------------------------------------------|
| منفصل   | كلي      | كل كيان في طبقة فرعية واحدة فقط، وكل كيان في الطبقة العليا يجب أن ينتمي إلى إحداها     |
| منفصل   | جزئي     | كل كيان في طبقة فرعية واحدة كحد أقصى، ولكن بعض الكيانات قد لا تنتمي لأي طبقة           |
| متداخل  | كلي      | الكيان قد ينتمي لأكثر من طبقة، وكل كيان في الطبقة العليا يجب أن ينتمي لواحدة على الأقل |
| متداخل  | جزئي     | الكيان قد ينتمي لأكثر من طبقة، أو لا ينتمي لأي واحدة                                   |

💡 التعميم غالباً ما يكون كلياً لأن الطبقة العليا مشتقة من الطبقات الفرعية.

## 7. التسلسلات والتداخل (Hierarchies and Lattices)

### أ. التسلسل الهرمي (Hierarchy)

- كل طبقة فرعية لها طبقة عليا واحدة فقط → تُسمى وراثته أحادية (Single Inheritance).
- الشكل البياني: شجرة.

### ب. الشبكة (Lattice)

- يمكن للفئة الفرعية أن تكون تابعة لأكثر من فئة عليا → تُسمى وراثته متعددة (Multiple Inheritance).
- تُعرف أيضًا بفئة مشتركة (Shared Subclass).

### مثال على شبكة (Lattice)

- Engineering\_Manager:
- فرعية من MANAGER (لأنه مدير)
- وفرعية من ENGINEER (لأنه مهندس)
- إذا: يرث صفات من كلا الفئتين.

✓ الوراثته تتضمن جميع السمات والعلاقات من كل الفئات العليا السابقة.

## 8. الأصناف (Categories) أو أنواع الاتحاد (Union Types)

### المفهوم

- حالة خاصة من العلاقة بين الفئة العليا والفرعية.
- الفئة الفرعية لها أكثر من فئة عليا، لكنها تمثل اتحادًا (Union) وليس تقاطعًا.

### الفرق بين الفئة المشتركة (Shared Subclass) ونوع الاتحاد (Category)

| المقارنة | الفئة المشتركة (Shared Subclass)                | نوع الاتحاد (Category)                                       |
|----------|-------------------------------------------------|--------------------------------------------------------------|
| العضوية  | يجب أن يكون الكيان عضوًا في كل الفئات العليا    | يجب أن يكون الكيان عضوًا في واحدة على الأقل من الفئات العليا |
| العلاقة  | تقاطع (Intersection)                            | اتحاد (Union)                                                |
| المثال   | يجب أن يكون مديرًا Engineering_Manager ومهندسًا | يمكن أن يكون شخصًا أو شركة أو مصرفًا OWNER                   |

### مثال تطبيقي: نظام تسجيل المركبات

- نريد نمذجة مالك المركبة (OWNER).

- المالك يمكن أن يكون:

- **PERSON** (فرد)
- **COMPANY** (شركة)
- **BANK** (مصرف – إذا كان هناك رهن)
- نُنشئ فئة فرعية تسمى **OWNER**، وهي نوع اتحاد من PERSON، COMPANY، BANK.
- أي:  $OWNER \subseteq PERSON \cup COMPANY \cup BANK$

💡 هذا يختلف عن الفئة المشتركة التي تتطلب أن يكون الكيان في جميع الفئات العليا.

## 9. ملخص المفاهيم الأساسية

| المفهوم                  | الوصف                                                      |
|--------------------------|------------------------------------------------------------|
| الطبقة العليا / الفرعية  | تمثيل التسلسل الهرمي للكيانات (مثلاً: EMPLOYEE → ENGINEER) |
| علاقة IS-A               | توضح أن "الفرعي هو نوع من الأعلى"                          |
| التخصص (Specialization)  | من العام إلى الخاص (Top-down)                              |
| التعميم (Generalization) | من الخاص إلى العام (Bottom-up)                             |
| الوراثة                  | الفئة الفرعية ترث السمات والعلاقات من العليا               |
| القيود                   | منفصل/متداخل، كلي/جزئي                                     |
| الشبكة (Lattice)         | وراثة متعددة من أكثر من فئة عليا                           |
| نوع الاتحاد (Category)   | فئة فرعية من اتحاد فئات عليا مختلفة                        |

## ملاحظات وأمثلة إضافية

### ✓ متى نستخدم التخصص؟

- عندما نريد تمييز كيانات بناءً على خصائص محددة.
- مثال: في نظام مستشفى:
- PATIENT → {INPATIENT, OUTPATIENT}
- EMPLOYEE → {DOCTOR, NURSE, ADMIN}

### ✓ متى نستخدم نوع الاتحاد (Category)؟

- عندما تكون الفئة الفرعية تمثل نوعاً من عدة أنواع مختلفة.
- مثال: في نظام تأمين:
- INSURED\_ENTITY → PERSON أو VEHICLE أو PROPERTY يمكن أن يكون
- نُنشئ فئة UNION تسمى INSURED

## كيف نرسم ذلك في EER Diagram? ✓

- الطبقة العليا والفرعية: مستطيلات متصلة بخط.
- نوع السهم: لا يُستخدم غالبًا (لعدم الضرورة).
- القيود:
- للتباعد (o) أو (d).
- خط واحد (جزئي) أو مزدوج (كلي) للاكتمال.
- نوع الاتحاد: يُرسم بخط منقط أو بعلامة خاصة (حسب النظام).

## الخلاصة

نموذج EER يوسع من قدرات نموذج ER التقليدي ليسمح بتمثيل أكثر دقة للواقع المعقد للبيانات. من خلال مفاهيم مثل التخصص، التعميم، الوراثة، والقيود، يمكن بناء نماذج بيانات قوية ومرنة تناسب قواعد البيانات الحديثة.

📌 **نصيحة:** استخدم التخصص عندما ترغب في تصنيف كيانات داخل نوع واحد، واستخدم نوع الاتحاد (Category) عندما يكون الكيان ناتجًا عن اتحاد أنواع مختلفة.

## Summary in English

# Chapter 4: Enhanced Entity-Relationship (EER) Modeling

## Introduction

The **Enhanced Entity-Relationship (EER)** model is an extension of the basic **Entity-Relationship (ER)** model introduced in Chapter 3. It adds more advanced modeling concepts to represent complex real-world situations more accurately, especially in large or sophisticated database systems.

### ✓ What does EER add to ER?

- Subclasses and Superclasses
- Specialization and Generalization
- Attribute and Relationship Inheritance
- Constraints on Specialization/Generalization
- Categories (Union Types)

These enhancements allow for richer semantic representation and better organization of entity types.

---

# 1. Subclasses and Superclasses

## Concept

An entity type may have meaningful subgroupings of its entities based on shared characteristics.

## Example

The entity type **EMPLOYEE** can be subdivided into:

- **SECRETARY, ENGINEER, TECHNICIAN** — based on *job type*
- **MANAGER** — based on *role*
- **SALARIED\_EMPLOYEE, HOURLY\_EMPLOYEE** — based on *payment method*

Each of these subgroupings is a **subclass** of **EMPLOYEE**, while **EMPLOYEE** is the **superclass**.

## Superclass/Subclass Relationship

- Also known as **IS-A relationships**:
  - **SECRETARY IS-A EMPLOYEE**
  - **ENGINEER IS-A EMPLOYEE**
  - **MANAGER IS-A EMPLOYEE**

## Key Points

- An entity in a subclass represents the **same real-world object** as in the superclass, but in a **specific role**.
- An entity **cannot exist solely** as a member of a subclass — it **must also be a member** of the superclass.
- A superclass entity **may or may not belong** to any of its subclasses.
- An entity can belong to **multiple subclasses simultaneously**.

## Examples

- A **salaried engineer** belongs to both:
  - **SALARIED\_EMPLOYEE** and **ENGINEER**
- An **engineering manager who is salaried** belongs to:
  - **MANAGER** , **ENGINEER** , and **SALARIED\_EMPLOYEE**



✦ **Note:** Not every EMPLOYEE needs to be in a subclass.

---

## 2. Specialization

### Definition

**Specialization** is the process of defining **subclasses** of a superclass based on distinguishing characteristics of the entities.

It is a **top-down** process: starting from a general entity and creating more specific types.

### Examples

1. Specialization of **EMPLOYEE** by **job type**:
  - {SECRETARY, ENGINEER, TECHNICIAN}
2. Specialization of **EMPLOYEE** by **role**:
  - {MANAGER}
3. Specialization of **EMPLOYEE** by **payment method**:
  - {SALARIED\_EMPLOYEE, HOURLY\_EMPLOYEE}

✅ A single superclass can have **multiple specializations**.

### Specific (Local) Attributes and Relationships

- Subclasses can have **attributes unique** to them:
    - Example: TypingSpeed for SECRETARY
  - Subclasses can participate in **specific relationships**:
    - Example: HOURLY\_EMPLOYEE may have a relationship BELONGS\_TO a ShiftSchedule
- 

## 3. Generalization

### Definition

**Generalization** is the **reverse** of specialization. It is a **bottom-up** process where several entity types with common attributes are **grouped into a more general superclass**.

### Example

- CAR and TRUCK → generalized into VEHICLE
- CAR and TRUCK become **subclasses** of VEHICLE

✓ You can view:

- {CAR, TRUCK} as a **specialization** of VEHICLE
- VEHICLE as a **generalization** of CAR and TRUCK

## Diagram Notation

- Some notations use arrows:
  - Arrow **to superclass** → generalization
  - Arrow **to subclasses** → specialization
- However, the distinction is often subjective.
- **Best practice:** Do **not** use arrows — treat both processes as leading to the same result.

## 4. Attribute and Relationship Inheritance

### Inheritance

An entity in a subclass **inherits**:

- All **attributes** from its superclass
- All **relationships** the superclass participates in

### Example

- SECRETARY inherits from EMPLOYEE :
  - Attributes: Name , SSN , Address , Salary , etc.
  - Relationships: WORKS\_FOR , MANAGES , etc.

✓ So every SECRETARY has a name, SSN, and works for a department — even if not explicitly shown in the subclass.

## 5. Types of Specialization

| Type                                             | Description                                                                         | Example                                           |
|--------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------|
| <b>Predicate-defined<br/>(Condition-defined)</b> | Membership in a subclass is determined by a condition (e.g., value of an attribute) | ENGINEER if JobType = "Engineer"                  |
| <b>Attribute-defined</b>                         | The defining condition is based on a single attribute of the superclass; the        | JobType defines {SECRETARY, ENGINEER, TECHNICIAN} |

| Type         | Description                                                         | Example                                                            |
|--------------|---------------------------------------------------------------------|--------------------------------------------------------------------|
|              | attribute name is shown in the diagram                              |                                                                    |
| User-defined | No automatic condition; membership is assigned manually by the user | Adding an employee to "Top Performer" category based on evaluation |

💡 If all subclasses in a specialization use the **same attribute** for membership, it's called an **attribute-defined specialization**, and that attribute is the **defining attribute**.

## 6. Constraints on Specialization/Generalization

### A. Disjointness Constraint

Determines whether subclasses are **mutually exclusive**.

- ✅ **Disjoint (d):**
  - An entity can be in **at most one** subclass.
  - Example: An employee cannot be both `SECRETARY` and `ENGINEER` at the same time.
- ❌ **Overlapping (o):**
  - An entity **can belong to more than one** subclass.
  - Example: An employee can be both `MANAGER` and `ENGINEER`.

✦ Labeled as **(d)** or **(o)** in EER diagrams.

### B. Completeness (Exhaustiveness) Constraint

Determines whether **every** superclass entity must belong to a subclass.

- ✅ **Total (double line):**
  - Every entity in the superclass **must** belong to **at least one** subclass.
  - Example: Every `EMPLOYEE` must be either `SALARIED` or `HOURLY`.
- ❌ **Partial (single line):**
  - Entities in the superclass **may not belong** to any subclass.
  - Example: Some employees may not be classified into job roles yet.

✦ Total completeness is common in **generalization**, as the superclass is derived from the subclasses.

# Four Types of Specialization/Generalization

| Disjointness | Completeness | Meaning                                                                           |
|--------------|--------------|-----------------------------------------------------------------------------------|
| Disjoint     | Total        | Each entity in exactly one subclass; all superclass entities are classified       |
| Disjoint     | Partial      | Each entity in at most one subclass; some may not be classified                   |
| Overlapping  | Total        | Entity may be in multiple subclasses; all superclass entities are in at least one |
| Overlapping  | Partial      | Entity may be in multiple subclasses; some may not be in any                      |

---

## 7. Specialization Hierarchies and Lattices

### A. Hierarchy (Single Inheritance)

- Each subclass has **only one superclass**.
- Forms a **tree-like structure**.
- Also called **single inheritance**.

### B. Lattice (Multiple Inheritance)

- A subclass can have **more than one superclass**.
- Called a **shared subclass**.
- Represents **multiple inheritance**.

### Example: Shared Subclass

- `ENGINEERING_MANAGER` is a subclass of both:
  - `MANAGER` (role-based)
  - `ENGINEER` (job-based)
- Therefore, it **inherits attributes and relationships from both**.

✔ In a lattice, a subclass inherits from **all its superclasses and their ancestors**.

---

## 8. Categories (Union Types)

### Concept

A **category** (or **union type**) is a subclass that has **more than one superclass**, but unlike a shared subclass, it represents the **union** of its superclasses — not their intersection.

## Key Difference

| Feature    | Shared Subclass                                       | Category (Union Type)                            |
|------------|-------------------------------------------------------|--------------------------------------------------|
| Membership | Must be member of <b>all</b> superclasses             | Must be member of <b>at least one</b> superclass |
| Set Theory | Subset of <b>intersection</b>                         | Subset of <b>union</b>                           |
| Example    | ENGINEERING_MANAGER must be both MANAGER and ENGINEER | OWNER can be a PERSON OR a COMPANY OR a BANK     |

## Real-World Example: Vehicle Registration

- **OWNER** can be:
    - A **PERSON**
    - A **COMPANY**
    - A **BANK** (holding a lien)
  - The category **OWNER** is defined as a subset of:
    - **PERSON**  $\cup$  **COMPANY**  $\cup$  **BANK**
  - An entity (e.g., John) can be an **OWNER** because he is a **PERSON** , even if he is not a **COMPANY** .
-  This is **not multiple inheritance** — it's a **union** of different entity types.

## 9. Summary of Key Concepts

| Concept                 | Description                                                                  |
|-------------------------|------------------------------------------------------------------------------|
| Subclass/Superclass     | A specialized/generalized entity relationship (e.g., ENGINEER IS-A EMPLOYEE) |
| Specialization          | Top-down process: creating subclasses from a superclass                      |
| Generalization          | Bottom-up process: creating a superclass from similar entities               |
| Inheritance             | Subclass inherits attributes and relationships from superclass               |
| Disjointness Constraint | Whether subclasses are mutually exclusive (d) or overlapping (o)             |
| Completeness Constraint | Whether all superclass entities must be in a subclass (total/partial)        |
| Hierarchy               | Tree structure with single inheritance                                       |

| Concept                      | Description                                                                            |
|------------------------------|----------------------------------------------------------------------------------------|
| <b>Lattice</b>               | Network structure with multiple inheritance (shared subclass)                          |
| <b>Category (Union Type)</b> | Subclass from union of different superclasses (e.g., OWNER from PERSON, COMPANY, BANK) |

## Additional Notes and Examples

### ✓ When to Use Specialization?

- When entities of the same type play **different roles**.
- Example: In a university:
  - PERSON → {STUDENT, FACULTY, STAFF}

### ✓ When to Use a Category (Union Type)?

- When the subclass comes from **different kinds of entities**.
- Example: In insurance:
  - INSURED\_ENTITY → can be PERSON, VEHICLE, or PROPERTY
  - Use a **category** to model this union.

### ✓ Diagramming Tips\*\*

- Represent subclasses and superclasses as **rectangles** connected by lines.
- Use **(d)** or **(o)** for disjointness.
- Use **single line** (partial) or **double line** (total) for completeness.
- For **attribute-defined specialization**, write the **defining attribute** (e.g., JobType) next to the line.
- For **categories**, use a **dotted line** or special notation (varies by convention).

## Conclusion

The **EER model** significantly enhances the expressive power of the basic ER model. By introducing **subclasses**, **inheritance**, **constraints**, and **union types**, it allows database designers to model complex real-world scenarios with greater precision and clarity.

✚ **Design Tip:** Use **specialization** for role-based distinctions within the same entity type, and **categories** when combining different entity types into a single logical group.

